



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - Sede Bogotá

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas e Industrial

Asignatura:

Computación Visual

Práctica 2

Sergio David Lopez Pardo

Carlos Javier Camacho Cely

Juan Daniel Ramirez Mojica

Juan Sebastian Munoz Lemus

Profesor:

Arbey Aragon Bohorquez

2025 - I

Bogotá D.C.

Índice

Fase 1 – Estudio Conceptual y Demostración Visual	3
1. SLAM - 5 minutos con Cyrill	3
2. Comprensión de SLAM (localización y mapeo simultáneos)	4
3. ¡3D sin cámaras! NeRF con texto e IA generativa (DotCSV)	5
4. ¡Me he CAPTURADO EN 3D con NeRF ...en SEGUNDOS! (DotCSV)	5
5. ¡NeRF MEJORADO! Controla escenas 3D con una IA (DotCSV)	6
6. ¡3D a partir de un SOLO FRAME! (DotCSV)	6
7. Una NUEVA REVOLUCIÓN en el 3D GAUSSIAN SPLATTING (DotCSV)	7
Fase 2 – Implementación de NeRF	7
1. Configuración del Entorno	7
2. Bibliotecas principales:	8
3. Dataset Usado	8
4. Tiempo de Entrenamiento	9
5. Calidad de Reconstrucción	10
6. Visualización Interactiva	10
7. Video 360°	11
8. Cambios y Correcciones Realizadas	12
9. Conclusiones	12
Fase 3 – Implementación de Gaussian Splatting	13
1. Objetivo	13
2. Configuración del Entorno	13
3. Problemas Encontrados	13
4. Soluciones Intentadas	14
5. Resultado	15
6. Conclusión	15
Fase 4 – Implementación de SLAM	15
Fase 5 – Comparación y Reflexión Final	16

Fase 1 – Estudio Conceptual y Demostración Visual

1. SLAM - 5 minutos con Cyrill

¿Qué problema resuelve?

SLAM (Simultaneous Localization and Mapping) resuelve el problema fundamental de que un robot o sensor móvil pueda construir un mapa del entorno mientras se localiza simultáneamente dentro de ese mismo mapa. Esto es crucial para moverse en espacios desconocidos o parcialmente conocidos, como ocurre con robots autónomos, vehículos autónomos o drones. Si ya se tiene un mapa, la localización es más sencilla, y si ya se tiene una localización precisa, mapear es más fácil; el reto está en hacer ambas tareas al mismo tiempo, lo cual representa un problema complejo de estimación y optimización.

¿Qué técnicas o modelos usa y cuál es el resultado final?

El video divide SLAM en dos componentes principales:

- Front-end: transforma los datos crudos del sensor en representaciones intermedias (como restricciones o distribuciones de probabilidad), por ejemplo, derivando la ubicación de un "landmark" detectado.
- Back-end: toma esa representación intermedia y resuelve el problema de estimación del estado mediante técnicas de optimización.

En el back-end, las técnicas más comunes incluyen:

- Filtros de Kalman extendidos (EKF)
- Filtros de partículas
- Aproximaciones basadas en mínimos cuadrados o gráficas (Graph-based SLAM)

Actualmente, los métodos más populares son los Graph-based SLAM, que representan las variables (como poses del robot o sensor) como nodos de un grafo, y las relaciones (como restricciones espaciales) como aristas. Estas gráficas pueden ser:

- Pose graphs, donde solo se representan las poses y se marginaliza el mapa.
- Factor graphs, que almacenan información como vectores entre los nodos.

Para resolver la optimización se utilizan bibliotecas como g2o, GTSAM o Ceres, las cuales minimizan el error global entre las mediciones y el modelo esperado (problema de mínimos cuadrados). El resultado final es una reconstrucción

coherente del entorno y una trayectoria optimizada del robot o sensor dentro de ese mapa.

2. Comprensión de SLAM (localización y mapeo simultáneos)

¿Qué problema resuelve?

En el video, se menciona que SLAM permite que un agente (cámara, robot, dron, etc.) construya un mapa de un entorno desconocido mientras estima su propia posición dentro de ese entorno, en tiempo real. Es fundamental para aplicaciones en las que no se tiene un mapa previo y se requiere navegación autónoma precisa, como en realidad aumentada, vehículos autónomos, drones, robótica de consumo y videojuegos. En contraste con la simple odometría visual, SLAM busca ofrecer una solución global, robusta y corregida en el tiempo, capaz de manejar errores acumulativos mediante realimentación (feedback).

¿Qué técnicas o modelos usa y cuál es el resultado final?

El proceso de SLAM se explica como un flujo modular con las siguientes etapas:

- Entrada de sensores: Puede ser cámara RGB, estéreo, radar, LiDAR, IMU, o incluso sensores barométricos.
- Extracción de características (feature extraction): Detecta puntos relevantes como esquinas y bordes usando algoritmos como SIFT, SURF, ORB, etc.
- Matching y estimación de pose (visual odometry): Rastrea cómo se mueven los puntos clave entre cuadros sucesivos para estimar el movimiento del agente.
- Loop closure: Detecta si el agente ha regresado a una posición ya visitada, lo cual permite reducir la deriva acumulativa en la estimación.
- Bundle Adjustment: Refina globalmente la trayectoria y la reconstrucción para minimizar errores.

Además, el sistema incorpora realimentación (feedback) desde loop closure y bundle adjustment hacia la estimación de pose, lo que incrementa la precisión. Se menciona también la diferencia con VIO (Visual Inertial Odometry), que es una solución más local, sin realimentación ni mapa global, pero más ligera computacionalmente.

En cuanto al rendimiento, se destaca la evolución desde CPUs y GPUs hacia DSPs y aceleradores hardware, buscando una mayor eficiencia energética para plataformas embebidas (como teléfonos o drones). Sin embargo, como los

algoritmos de SLAM están en constante cambio, se recomienda mantener cierta flexibilidad mediante procesamiento programable.

Resultado final:

Un mapa 3D (o 2D en ciertos casos) del entorno, junto con una estimación precisa y en tiempo real de la trayectoria del agente. Esta información puede usarse para navegación autónoma, realidad aumentada, reconstrucción de escenas, y más.

3. ¡3D sin cámaras! NeRF con texto e IA generativa (DotCSV)

¿Qué problema resuelve?

Este trabajo aborda dos grandes limitaciones de los NeRF clásicos:

- a. Su incapacidad para manejar variaciones fotométricas (cambios en iluminación, clima o cámara).
- b. Su fragilidad frente a elementos transitorios (personas, autos, andamios que cambian entre imágenes).

NeRF tradicional supone que todas las imágenes provienen de una escena estática y homogénea, lo cual no representa la realidad del mundo exterior, especialmente cuando se usan imágenes de diferentes turistas o cámaras.

¿Qué técnicas o modelos usa y cuál es el resultado final?

“NeRF in the Wild” propone una arquitectura mejorada que separa el modelado de la escena en dos fases: una para la estructura estática (lo importante, como monumentos) y otra para los elementos transitorios (personas, coches, sombras cambiantes), incorporando además vectores de embeddings que codifican el estilo y condiciones lumínicas de cada imagen. El resultado es una reconstrucción 3D fotorealista, flexible y robusta a condiciones cambiantes, con posibilidad de generar nuevas vistas y modificar la estética de la escena (por ejemplo, ver un monumento al atardecer o con otro estilo de iluminación). Esta evolución permite reconstruir lugares como la Fontana di Trevi o la Torre Eiffel usando fotos turísticas de internet.

4. ¡Me he CAPTURADO EN 3D con NeRF ...en SEGUNDOS! (DotCSV)

¿Qué problema resuelve?

El video aborda las limitaciones de la fotogrametría tradicional para generar escenas 3D realistas a partir de fotografías, especialmente en cuanto a capturar efectos de iluminación complejos como reflejos, transparencias o brillos. Estas técnicas clásicas generan mallas 3D huecas que fallan en reconstrucciones precisas cuando hay materiales brillantes o

condiciones lumínicas variadas. Además, los métodos basados en NeRF (renderizado volumétrico neuronal) solían tardar horas o días en generar escenas, lo que limitaba su aplicabilidad.

¿Qué técnicas o modelos usa y cuál es el resultado final?

Este nuevo sistema basado en NeRF utiliza renderizado volumétrico neural ultra-rápido, que permite reconstruir escenas 3D fotorrealistas en cuestión de segundos a partir de un simple video grabado con el móvil. En lugar de modelar superficies, predice la densidad y color de cada punto tridimensional en función de su posición y la orientación de la cámara, capturando incluso fenómenos ópticos como distorsiones a través de gafas o reflejos. Comparado con fotogrametría (ej. Meshroom), NeRF logra resultados más detallados y fieles, aunque con resolución variable. La tecnología permite ahora crear versiones 3D de entornos reales y hasta de personas (como el “nerfi” del propio youtuberDot CSV) de manera interactiva, rápida y sin necesidad de equipamiento especializado.

5. ¡NeRF MEJORADO! Controla escenas 3D con una IA (DotCSV)

¿Qué problema resuelve?

El video presenta el problema de cómo generar escenas 3D hiperrealistas navegables sin depender de modelos geométricos clásicos como mallas o polígonos, que suelen fallar al representar superficies reflectantes, brillos o complejidades lumínicas. Además, técnicas como el ray tracing, aunque precisas, son costosas en tiempo de cómputo y memoria. Frente a estas limitaciones, se plantea una alternativa más eficiente y precisa mediante una técnica basada en inteligencia artificial que permite interpolar nuevas vistas realistas a partir de imágenes reales.

¿Qué técnicas o modelos usa? ¿Cuál es el resultado final?

La solución presentada es NeRF (Neural Radiance Fields), que combina el método volume ray marching con una red neuronal multicapa que reemplaza la tradicional función de transferencia. Esta red, entrenada con imágenes desde distintos ángulos, predice para cada punto del espacio el color y opacidad adecuados según su posición y dirección del rayo. El resultado final es un modelo extremadamente compacto (solo 5 MB) que permite renderizar escenas 3D fotorrealistas desde cualquier perspectiva, logrando una representación visual que engaña al ojo humano y abre posibilidades en campos como la realidad virtual o los gráficos en tiempo real.

6. ¡3D a partir de un SOLO FRAME! (DotCSV)

¿Qué problema resuelve?

En este video se aborda el desafío de acelerar drásticamente el proceso de entrenamiento de escenas 3D con NeRF (Neural Radiance Fields), que tradicionalmente toma horas o incluso días. El problema surge de la necesidad de representar escenas 3D complejas con iluminación realista desde múltiples ángulos, pero con un costo computacional elevado. La solución planteada intenta permitir que una red neuronal pueda generar una reconstrucción

tridimensional navegable a partir de una única imagen (o muy pocas), en cuestión de segundos, sin perder calidad.

¿Qué técnicas o modelos usa? ¿Cuál es el resultado final?

Para lograrlo, se introduce una técnica optimizada por NVIDIA que combina NeRF con estructuras de datos hash en lugar de grillas tridimensionales tradicionales, permitiendo comprimir y organizar eficientemente la información espacial. Esto se complementa con una red neuronal que toma como entrada no solo las coordenadas 3D, sino también la dirección de observación. Gracias al uso de funciones hash para mapear regiones espaciales y permitir colisiones útiles (donde la red aprende qué información es más relevante), se logra entrenar una escena NeRF en segundos con una GPU RTX 3090, permitiendo reconstrucciones 3D detalladas y realistas casi en tiempo real. Además, la técnica se extiende a otras tareas como codificación de imágenes de alta resolución y geometrías complejas, abriendo la puerta a generar contenido 3D con IA de forma rápida e interactiva.

7. Una NUEVA REVOLUCIÓN en el 3D GAUSSIAN SPLATTING (DotCSV)

¿Qué problema resuelve?

Este video aborda la reconstrucción de escenas 3D realistas a partir de imágenes o videos, permitiendo mover libremente la cámara incluso en ángulos no capturados originalmente. Aunque técnicas anteriores como NeRF ya habían avanzado en este campo, presentaban limitaciones en calidad de fondo, tiempos de inferencia y eficiencia. Gaussian Splatting surge como una alternativa más rápida y precisa, resolviendo especialmente los problemas de renderización en tiempo real y representación explícita de escenas tridimensionales.

¿Qué técnicas o modelos usa? ¿Cuál es el resultado final?

La técnica se basa en usar distribuciones gaussianas tridimensionales como "manchas" o trazos que forman la escena 3D, cuyos parámetros (posición, forma, color, opacidad) son ajustados mediante backpropagation, como en el entrenamiento de redes neuronales. A diferencia de NeRF, aquí la escena sí se representa de forma explícita, lo que permite renderizar en más de 100 fps, incluso con escenas dinámicas (4D). Esto habilita funcionalidades como reencuadres de cámara, composición avanzada, tracking 3D preciso, y visualización en realidad virtual, convirtiéndolo en un sistema revolucionario para gráficos volumétricos realistas.

Fase 2 – Implementación de NeRF

1. Configuración del Entorno

El proyecto se ejecutó en Google Colab, aprovechando su entorno preconfigurado con acceso a GPU (cuando se habilita desde el menú de runtime). Se utilizó TensorFlow 2.x, ya

que TensorFlow 1.x no es compatible con las versiones actuales de Colab. Para asegurar compatibilidad con ciertas funciones, se habilitó la ejecución eager con:

```
Python
tf.compat.v1.enable_eager_execution()
```

2. Bibliotecas principales:

- TensorFlow 2.x – Para la construcción y entrenamiento del modelo NeRF.
- NumPy – Manejo de arreglos, imágenes y poses.
- Matplotlib – Visualización de imágenes renderizadas y gráficos de rendimiento (PSNR).
- tqdm.notebook – Progreso visual del entrenamiento.
- ImageIO – Generación del video final en formato .mp4.
- ipywidgets – Visualización interactiva de la escena 3D.

No fue necesaria instalación adicional, ya que todo está incluido por defecto en Colab.

3. Dataset Usado

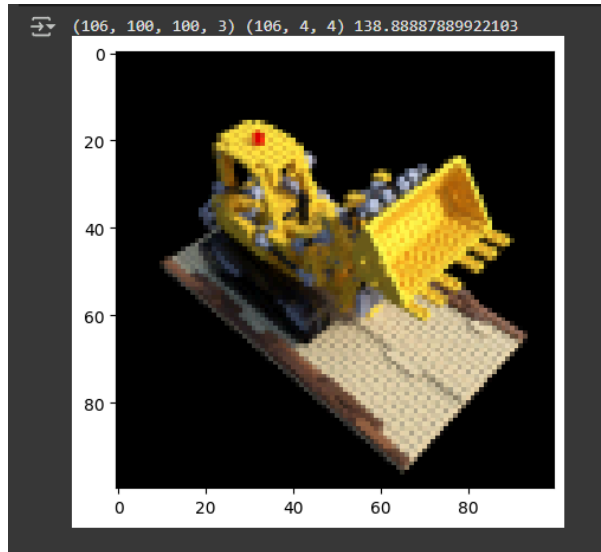
Se utilizó el dataset predefinido `tiny_nerf_data.npz`, descargado directamente desde:

http://cseweb.ucsd.edu/~viscomp/projects/LF/papers/ECCV20/nerf/tiny_nerf_data.npz

Este dataset contiene:

- images: 106 imágenes RGB de tamaño 100x100 px de una escena simple
- poses: Matrices de transformación de cámara correspondientes.
- focal: Parámetro focal de la cámara.

Se usaron las primeras 100 imágenes para el entrenamiento, y una imagen adicional como conjunto de prueba (testing y testpose).

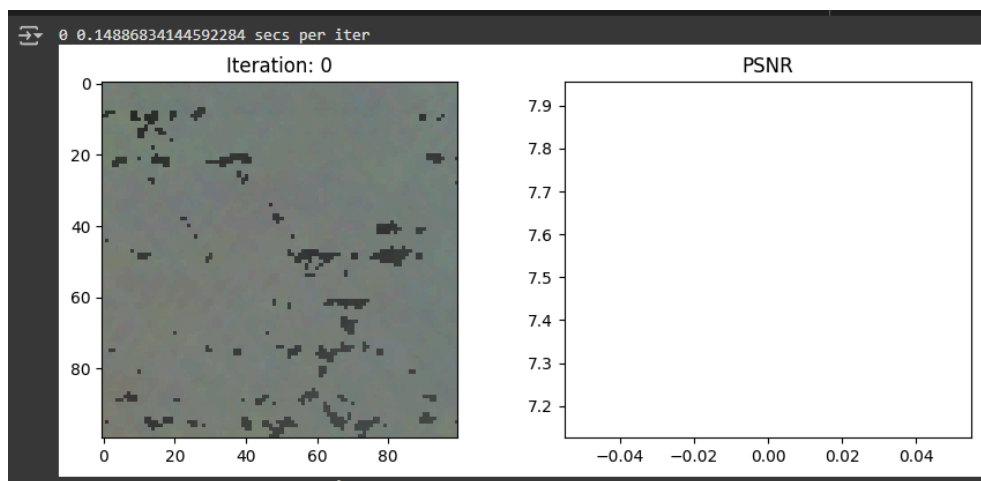


4. Tiempo de Entrenamiento

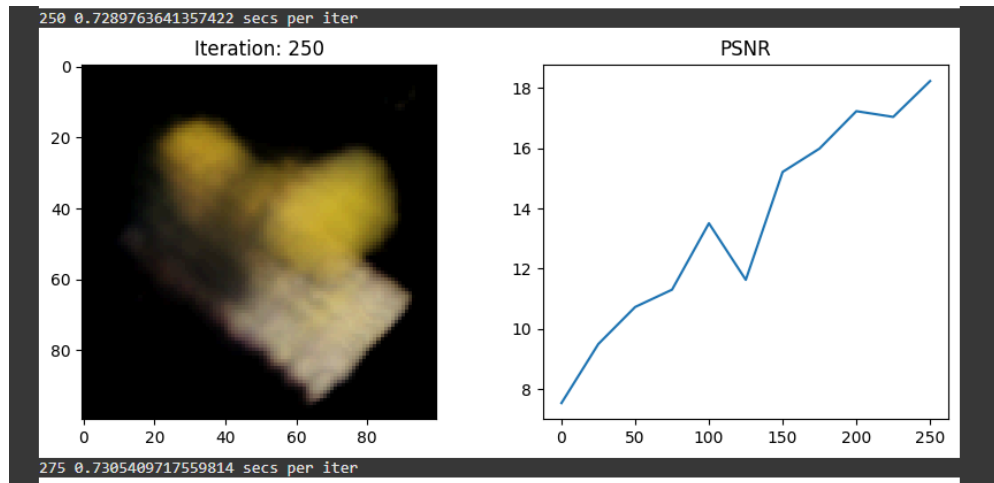
El modelo se entrenó durante 1000 iteraciones ($N_{\text{iters}} = 1000$), usando 64 muestras por rayo ($N_{\text{samples}} = 64$). Se evaluó el tiempo promedio de iteración cada 25 pasos:

- Tiempo promedio por iteración: ~ 0.74 segundos
- Tiempo total estimado: $1000 \times 0.74 \approx 740$ segundos ≈ 12.3 minutos

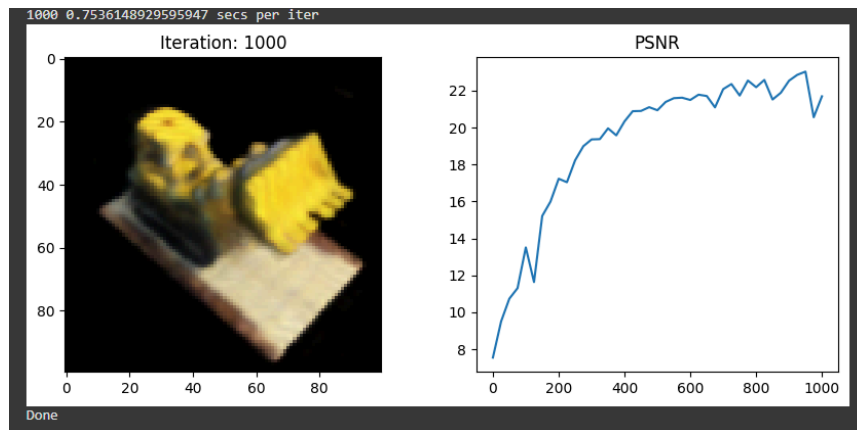
Iteración 1:



Iteración 250:



Iteración 1000:



5. Calidad de Reconstrucción

Cada 25 iteraciones se renderizó una imagen de prueba para evaluar el progreso visual y calcular el PSNR (Peak Signal-to-Noise Ratio). A lo largo del entrenamiento, se observó una mejora progresiva:

- Imagen renderizada: Mejora significativa cada 25 iteraciones.
- Gráfico de PSNR: Incremento sostenido, reflejando el aprendizaje del modelo.

6. Visualización Interactiva

Se incluyó una herramienta interactiva con ipywidgets para ajustar los parámetros theta, phi y radius, generando imágenes desde distintos ángulos esféricos.

Python

```
interactive_plot = interactive(f, {...})
```

Esta interfaz permite rotar la cámara y observar la escena desde distintas posiciones, ofreciendo una comprensión visual 3D más rica.

7. Video 360°

Se generó un video de 360 grados orbitando la escena reconstruida, renderizando 120 imágenes (3 fps).

Python

```
frames = []
for th in np.linspace(0., 360., 120):
    ...
imageio.mimwrite('video.mp4', frames, fps=30, quality=7)
```

Resultado: Un video .mp4 con buena calidad visual, mostrando una rotación fluida alrededor del objeto reconstruido.



8. Cambios y Correcciones Realizadas

Durante la ejecución, se realizaron varias correcciones al código original:

- **Error de TensorFlow 1.x**

Solución: Se eliminó la línea `%tensorflow_version 1.x` y se habilitó ejecución eager para compatibilidad.

- **ValueError en tf.keras.Input**

Problema: Se pasaba un entero en lugar de una tupla.

Solución:

```
inputs = tf.keras.Input(shape=(3 + 3*2*L_embed,))
```

- **Error al usar tf.concat con KerasTensors**

Solución: Se usó `tf.keras.layers.concatenate` en lugar de `tf.concat`.

9. Conclusiones

- El modelo Tiny-NeRF permitió entender los principios básicos de reconstrucción NeRF en un entorno controlado y rápido.
- La calidad de reconstrucción fue buena, mostrando mejoras visuales y métricas (PSNR) consistentes.
- Las visualizaciones 3D y el video 360° ofrecieron una forma clara de explorar los resultados.

Fase 3 – Implementación de Gaussian Splatting

1. Objetivo

Explorar el método de reconstrucción 3D Gaussian Splatting, utilizando una implementación funcional en Google Colab basada en el repositorio `camenduru/gaussian-splatting-colab`, que internamente usa el código oficial `graphdeco-inria/gaussian-splatting`.

2. Configuración del Entorno

El entorno se configuró en Google Colab, intentando ejecutar los siguientes pasos:

```
Python
!git clone --recursive https://github.com/camenduru/gaussian-splatting
```

a. Instalar dependencias:

```
Python
!pip install -q plyfile
!pip install -q /content/gaussian-splatting/submodules/diff-gaussian-rasterization
!pip install -q /content/gaussian-splatting/submodules/simple-knn
```

b. Descargar un dataset de ejemplo (tandt):

```
Python
!wget https://huggingface.co/camenduru/gaussian-splatting/resolve/main/tandt_db.zip
!unzip tandt_db.zip
```

c. Ejecutar el entrenamiento:

```
Python
!python train.py -s /content/gaussian-splatting/tandt/train
```

3. Problemas Encontrados

Durante el proceso se encontraron múltiples errores al intentar compilar los submódulos esenciales para el funcionamiento del entrenamiento:

a. Error al instalar diff_gaussian_rasterization

```
Python
Obtaining file:///content/gaussian-splatting/submodules/diff-gaussian-rasterization
Installing collected packages: diff_gaussian_rasterization
  Running setup.py develop for diff_gaussian_rasterization
    error: subprocess-exited-with-error
      × python setup.py develop did not run successfully.
        | exit code: 1
```

b. Error al instalar simple-knn

```
Python
Obtaining file:///content/gaussian-splatting/submodules/simple-knn
Installing collected packages: simple_knn
Running setup.py develop for simple_knn
error: subprocess-exited-with-error
  × python setup.py develop did not run successfully.
  | exit code: 1
```

4. Soluciones Intentadas

Se intentaron múltiples soluciones para corregir el problema:

a. Instalación de herramientas de compilación:

```
Python
!sudo apt-get install -y ninja-build
!pip install ninja
```

b. Reinstalación usando pip install -e:

```
Python
!pip install -e submodules/diff-gaussian-rasterization
!pip install -e submodules/simple-knn
```

c. Verificación manual de rutas y dependencias

A pesar de estas acciones, los errores persistieron. El entorno Colab no logró compilar correctamente los módulos nativos requeridos, posiblemente por incompatibilidades con las versiones del compilador o el sistema operativo del entorno virtual.

Aunque se logró descargar correctamente el dataset de prueba tandt/truck, no se pudo continuar con el entrenamiento ni generar los splats, ya que el código dependía de los módulos que fallaron en compilarse.

Contenido descargado:

```
Python
tandt/truck/images/000001.jpg ... 000251.jpg
tandt/truck/sparse/0/{cameras.bin, images.bin, points3D.bin, project.ini}
```

5. Resultado

No fue posible completar el entrenamiento ni generar una visualización, debido a errores críticos en la instalación de los módulos C++/CUDA.

6. Conclusión

Gaussian Splatting representa un método avanzado y eficiente para reconstrucción 3D, pero su implementación actual depende de módulos que requieren compilación nativa, lo cual no es completamente compatible con entornos virtualizados como Colab.

Para lograr una instalación funcional se recomienda:

Ejecutar en un entorno local con GPU, como una máquina con CUDA 11.x, Python 3.10 y acceso root.

Usar una implementación alternativa como EasyGaussianSplatting, que simplifica la configuración.

Fase 4 – Implementación de SLAM

Como primera alternativa para la Fase 4, intenté trabajar con el cuaderno de Google Colab proporcionado por el libro GTBook, específicamente: S74_drone_perception.ipynb.

Este cuaderno estaba diseñado para simular sensores inerciales (IMU) y realizar reconstrucción 3D con imágenes mediante Structure from Motion (SfM), cumpliendo con los objetivos de la fase. Sin embargo, no fue posible ejecutarlo correctamente, ya que el entorno de ejecución de Colab fallaba constantemente, mostrando el mensaje:

```
Python
"Your session crashed for an unknown reason."
```

Este fallo ocurría incluso después de seguir los pasos recomendados, como instalar la librería gtbook y reiniciar el entorno. A pesar de intentarlo en múltiples sesiones y con distintos entornos de ejecución, el kernel se reiniciaba automáticamente tras unos segundos de ejecución.

Los mensajes de error más relevantes en los logs de Colab fueron:

```
Python
Jul 1, 2025, 3:27:51 PM WARNING kernel b3fdceb5-4155-436f-b6ba-285308bbe745
restarted
Jul 1, 2025, 3:27:51 PM INFO KernelRestarter: restarting kernel (1/5), keep
random ports
Jul 11, 2025, 3:27:21 PM WARNING Got events for closed stream
<zmq.eventloop.zmqstream.ZMQStream object>
Jul 11, 2025, 3:27:21 PM WARNING zmq message arrived on closed channel
Jul 11, 2025, 3:27:07 PM WARNING Debugger warning: It seems that frozen modules
are being used...
```

Estas advertencias indicaban que el kernel se reiniciaba debido a conflictos internos en la configuración del entorno de ejecución de Python en Colab (posiblemente por incompatibilidades con `gtbook`, `gtsam` o módulos congelados).

Por lo tanto, **este cuaderno no pudo utilizarse para completar la fase**, y se procedió a intentar otras opciones alternativas para la implementación de SLAM.

Fase 5 – Comparación y Reflexión Final

Para esta fase se redactó un informe en formato Markdown (.md), el cual se encuentra dentro de la carpeta correspondiente a la Fase 5 del proyecto.

📁 El informe está disponible en el siguiente enlace: **PONER ENLACE A EL INFORME EN SUS REPOS**